

Chapter 11

Species Recognition in Flowers

Introduction

The conventional approach of the human person for plant classification is comparing the color and shape of the leaves, but by using Artificial Intelligence analysis can provide quicker and more detailed results by analyzing the morphology of the leaves and provides more details about the leaf properties. Species Recognition has been a very important part of agriculture and there are thousands of species in plants which can be identified by a single person and even if it does so It will take a huge lot of time to do it.

In this chapter we will discuss and built a project related to identify species in flower with an image. In this project we will be using Artificial Intelligence different branches like machine learning, deep learning and computer vision to build such an amazing project. So, let's begin

Installation of Packages

In this species Recognition we have used so many libraries which need be installed before implementing this project. The command for installing these libraries are

- Warnings- pip install pytest-warnings/warnings
- NumPy- pip install Numpy
- hPandas- pip install pandas
- Matplotlib- pip install matplotlib
- Seaborn- pip install seaborn
- Tqdm- pip install tqdm
- Pillow/PIL- pip install pillow
- TensorFlow- pip install TensorFlow
- Keras- pip Install keras

For Python IDE: Enter these provided commands on the command prompt in windows

For Anaconda: Enter these commands on Anaconda Prompt

If you are using any other Environment or IDE then you have to specify the path for python and you can do the installation.

This project was implemented in Google Research Collaboratory. And you can download the copy of the code form this link_____

Dataset

In this project we are using a dataset from Kaggle which has 5 categories of species in flowers namely daisy, dandelion, rose, sunflower and tulip which totally consist of 4242 images with 320×240 pixels. The images in the dataset is based on the data flicr, google images and Yandex images. These images can be used to recognize the plant related to these species.

You can download the same dataset from this link _____

Implementation:

Note: This project is purely implemented on google research Collaboratory which you access by this link _____ and you are free to use any other IDE for project implementation.

Step 1: Importing all the Libraries

Note: The Symbol # represents the comment in the project which will make you understand about the code.

```
# Ignore the warnings
import warnings
warnings.filterwarnings ('always')
warnings.filterwarnings ('ignore')

# data visualisation and manipulation
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import style
import seaborn as sns

#configure
# sets matplotlib to inline and displays graphs below the
corresponding cell.
% matplotlib inline
style.use ('fivethirtyeight')
sns.set (style='whitegrid',color _ codes=True)

#model selection
from sklearn.model _ selection import train _ test _ split
from sklearn.model _ selection import KFold
from sklearn.metrics import accuracy _ score,precision _
score,recall _ score,confusion _ matrix,roc _ curve,
    roc _ auc _ score
from sklearn.model _ selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder
```

```
#preprocess.
from keras.preprocessing.image import ImageDataGenerator

#These are deep learning libraries which is present in keras
if you have installed keras these all will get installed
automatically.
from keras import backend as K
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam,SGD,Adagrad,Adadelta,RMSprop
from keras.utils import to_categorical

# we use these libraries specifically for CNN (convolutional
Neural Network) Building which will be used to use to recognize
the species of flower.
from keras.layers import Dropout, Flatten,Activation
from keras.layers import Conv2D, MaxPooling2D,
BatchNormalization

import tensorflow as tf
import random as rn

# here we are using OpenCV library to give input to the trained
model and specifically for manipulating zipped images and
getting NumPy arrays of pixel values of images.
import cv2
import numpy as np
from tqdm import tqdm
import os
from random import shuffle
from zipfile import ZipFile
from PIL import Image
```

Step 2: Mounting to drive for dataset

If you are using the google research Collaboratory then you can download the dataset from this link _____ and upload it in your google drive “Colab Notebooks” folder then you can link it by using this code

```
from google.colab import drive
drive.mount ('/gdrive')
```

Output:

Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=94.....y

Enter your authorization code:

.....

Mounted at /gdrive

This is the code for mounting your drive with google research Collaboratory which will ask you authentication which you can do it with your Gmail account by just following the simple steps. As you can see in the output how Collaboratory will ask you to verify it by clicking in the given link then it will get mounted by giving a authentication to it.

```
with open ('/gdrive/My Drive/foo.txt', 'w') as f:
f.write ('Hello Google Drive!')
!cat '/gdrive/My Drive/foo.txt'
```

Output:

Hello Google Drive!

This is the code which will help you to check whether your drive has been mounted with the Collaboratory or not.

After doing this we will access our dataset which has been uploaded in the drive directly and unzip it to access the files which we require in this project.

Step 3: Getting Data ready for Model

Here we are forming an array of images X and Y which will store all the images in the form of training and validation images for the CNN model to train and get tested with real data.

```
X=[]
Z=[]
IMG_SIZE=150 #assigning the image size constantly for the project
FLOWER_Daisy_DIR='drive/Colab Notebooks/flower recog/flowers/
daisy'
FLOWER_Sunflower_DIR='drive/Colab Notebooks/flower recog/
flowers/sunflower'
FLOWER_Tulip_DIR='drive/Colab Notebooks/flower recog/flowers/
tulip'
FLOWER_Dandi_DIR='drive/Colab Notebooks/flower recog/flowers/
dandelion'
FLOWER_Rose_DIR='drive/Colab Notebooks/flower recog/flowers/
rose'
```

Creating a function to label the images in the data which will be further used for training purpose.

```
def assign_label (img,flower_type):
return flower_type
```

here after labelling we will make a training data Separately from each categories of flower

```
def make_train_data (flower_type,DIR):
for img in tqdm (os.listdir (DIR)):
label=assign_label (img,flower_type)
```

```
path = os.path.join (DIR,img)
img = cv2.imread (path,cv2.IMREAD_COLOR)
img = cv2.resize (img, (IMG_SIZE,IMG_SIZE))

X.append (np.array (img))
Z.append (str (label))

# Training data for daisy
make_train_data ('Daisy',FLOWER_Daisy_DIR)
print (len (X))
```

Output:

```
100%|██████████| 769/769 [00:07<00:00, 101.93it/s]769
#Training data for sunflower
make_train_data ('Sunflower',FLOWER_Sunflower_DIR)
print (len (X))
```

Output:

```
100%|██████████| 766/766 [00:07<00:00, 176.83it/s]1535

#training Data for Tulip
make_train_data ('Tulip',FLOWER_Tulip_DIR)
print (len (X))
```

Output:

```
100%|██████████| 991/991 [00:08<00:00, 116.01it/s]2526

#training data for dandelion
make_train_data ('Dandelion',FLOWER_Dandelion_DIR)
print (len (X))
```

Output:

```
100%|██████████| 1052/1052 [00:09<00:00, 114.67it/s]3578

#training data for Rose
make_train_data ('Rose',FLOWER_Rose_DIR)
print (len (X))
```

Output:

```
100%|██████████| 784/784 [00:06<00:00, 116.67it/s]4362
```

This is how training data for all the flowers is ready for the model to get trained.

Step 4: Visualizing the data

After Preparing the data for training we will visualize the data we have prepared using matplotlib and opencv.

```
fig,ax=plt.subplots (5,2)
fig.set_size_inches (15,15)
for i in range (5):
    for j in range (2):
        l=mn.randint (0,len (Z))
        ax[i,j].imshow (X[l])
        ax[i,j].set_title ('Flower: '+Z[l])
plt.tight_layout ()
```

Output: as I Fig. 11.1

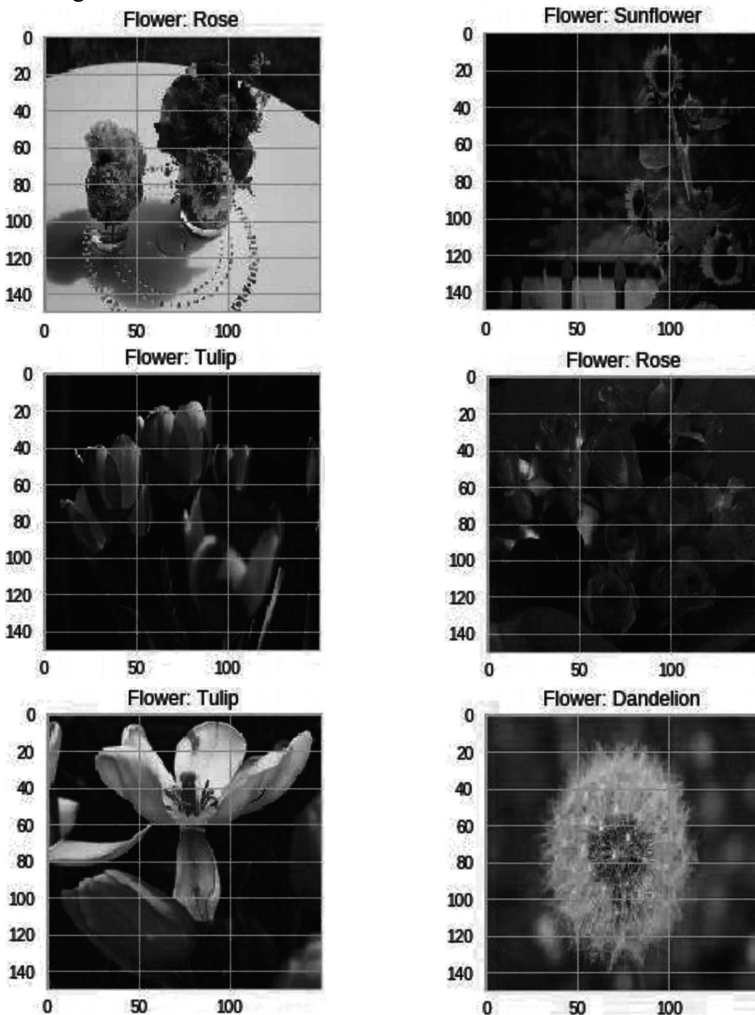


Fig. 11.1 Data visualization from dataset

Step-5: Labelling the data and splitting it into training and validation data

```
#labelling
le=LabelEncoder ()
Y=le.fit_transform (Z)
Y=to_categorical (Y,5)
X=np.array (X)
X=X/255

#After labelling we will do Splitting of data into Training and
Validation Sets
x_train,x_test,y_train,y_test=train_test_split (X,Y,test_
size=0.25,random_state=42)

#Setting the random seeds
np.random.seed (42)
rn.seed (42)
tf.set_random_seed (42)
```

Step-6: Building The model

For building the model I have used Keras Sequential in which it is very easy to build any model for which we have a lot of images as input to extract features and classify them.

In the model creation, I have created four convolutional layer, In the first layer we have 32 filters, 5×5 of kernel matrix with padding and an activation function ReLU with an input size of an image.

In the Second layer we have 64, filters, 3×3 of kernel matrix with padding and an activation function ReLU with an input size of an image.

In the Third and fourth layer we have 96, filters, 3×3 of kernel matrix with padding and an activation function ReLU with an input size of an image.

Following all these convolutional layers we will have a max pooling layer which will help us to get maximum features from the filters. In between the layers we have “flatten ()” layer which always server as the connection between the convolution and dense layer. And at last we have our fully connected output layers i.e., dense layers with an activation function.

```
# modelling starts using a CNN.

model = Sequential ()
model.add (Conv2D (filters = 32, kernel_size = (5,5),padding =
'Same',activation = 'relu', input_shape = (150,150,3)))
model.add (MaxPooling2D (pool_size= (2,2)))

model.add (Conv2D (filters = 64, kernel_size = (3,3),padding =
'Same',activation = 'relu'))
model.add (MaxPooling2D (pool_size= (2,2), strides= (2,2)))
```

```
model.add (Conv2D (filters =96, kernel_size = (3,3),padding =
'Same',activation ='relu'))
model.add (MaxPooling2D (pool_size= (2,2), strides= (2,2)))

model.add (Conv2D (filters = 96, kernel_size = (3,3),padding =
'Same',activation ='relu'))
model.add (MaxPooling2D (pool_size= (2,2), strides= (2,2)))

model.add (Flatten ())

model.add (Dense (512))
model.add (Activation ('relu'))
model.add (Dense (5, activation = "softmax"))
```

#after the model building we will declare our number of epochs and their batch size and we are using LR annealer for fixing our learning rate after a certain period of time while compiling the model.

```
#using a LR Annealer
batch_size=128
epochs=50
```

```
from keras.callbacks import ReduceLROnPlateau
red_lr= ReduceLROnPlateau (monitor='val_acc',
patience=3,verbose=1,factor=0.1)
```

#we are here performing Data Augmentation to prevent Overfitting

```
datagen = ImageDataGenerator (
featurewise_center=False, # set input mean to 0 over the dataset
samplewise_center=False, # set each sample mean to 0
featurewise_std_normalization=False, # divide inputs by std of the dataset
samplewise_std_normalization=False, # divide each input by its std
zca_whitening=False, # apply ZCA whitening
rotation_range=10, # randomly rotate images in the range (degrees, 0 to 180)
zoom_range = 0.1, # Randomly zoom image
width_shift_range=0.2, # randomly shift images horizontally (fraction of total width)
height_shift_range=0.2, # randomly shift images vertically (fraction of total height)
horizontal_flip=True, # randomly flip images
vertical_flip=False) # randomly flip images

datagen.fit (x_train)
```



```
#now we will do Compiling of our Model & Viewing Summary of Model
model.compile (optimizer=Adam (lr=0.001),loss='categorical_
crossentropy',metrics=['accuracy'])
model.summary ()
```

Output:

```
Layer (type) Output Shape Param #
=====
conv2d_1 (Conv2D) (None, 150, 150, 32) 2432
max_pooling2d_1 (MaxPooling2D) (None, 75, 75, 32) 0
conv2d_2 (Conv2D) (None, 75, 75, 64) 18496
max_pooling2d_2 (MaxPooling2D) (None, 37, 37, 64) 0
conv2d_3 (Conv2D) (None, 37, 37, 96) 55392
max_pooling2d_3 (MaxPooling2D) (None, 18, 18, 96) 0
conv2d_4 (Conv2D) (None, 18, 18, 96) 83040
max_pooling2d_4 (MaxPooling2D) (None, 9, 9, 96) 0
flatten_1 (Flatten) (None, 7776) 0
dense_1 (Dense) (None, 512) 3981824
activation_1 (Activation) (None, 512) 0
dense_2 (Dense) (None, 5) 2565
=====
Total params: 4,143,749
Trainable params: 4,143,749
Non-trainable params: 0
=====
```

Step 7: Training the Model with dataset

After the compilation of the model we will train our model with our testing data which we have labelled and separated earlier.

```
History = model.fit_generator (datagen.flow (x_train,y_train,
batch_size=batch_size),
epochs = epochs, validation_data = (x_test,y_test),
verbose = 1, steps_per_epoch=x_train.shape[0] // batch_size)
# model.fit (x_train,y_train,epochs=epochs,batch_
size=batch_size,validation_data = (x_test,y_test))
```

Output:

```
Epoch 1/50
25/25 [=====] - 23s 903ms/step - loss:
1.4743 - acc: 0.3588 - val_loss: 1.2462 - val_acc: 0.5069
Epoch 2/50
25/25 [=====] - 20s 808ms/step - loss:
1.1573 - acc: 0.5212 - val_loss: 1.1553 - val_acc: 0.5252
Epoch 3/50
25/25 [=====] - 20s 812ms/step - loss:
1.0605 - acc: 0.5745 - val_loss: 1.0670 - val_acc: 0.6013
Epoch 4/50
25/25 [=====] - 21s 820ms/step - loss:
1.0024 - acc: 0.5956 - val_loss: 0.9759 - val_acc: 0.6114
Epoch 5/50
25/25 [=====] - 20s 795ms/step - loss:
0.9267 - acc: 0.6437 - val_loss: 0.9823 - val_acc: 0.6205
Epoch 6/50
25/25 [=====] - 20s 784ms/step - loss:
0.9096 - acc: 0.6477 - val_loss: 0.9942 - val_acc: 0.6004
Epoch 7/50
25/25 [=====] - 20s 807ms/step - loss:
0.8594 - acc: 0.6677 - val_loss: 0.9092 - val_acc: 0.6581
Epoch 8/50
25/25 [=====] - 20s 812ms/step - loss:
0.8468 - acc: 0.6779 - val_loss: 0.8853 - val_acc: 0.6783
Epoch 9/50
25/25 [=====] - 20s 810ms/step - loss:
0.7789 - acc: 0.7019 - val_loss: 0.8087 - val_acc: 0.6856
Epoch 10/50
25/25 [=====] - 20s 798ms/step - loss:
0.7613 - acc: 0.7089 - val_loss: 0.8019 - val_acc: 0.6939
Epoch 11/50
25/25 [=====] - 20s 795ms/step - loss:
0.7660 - acc: 0.6997 - val_loss: 0.8029 - val_acc: 0.7030
Epoch 12/50
25/25 [=====] - 20s 794ms/step - loss:
0.7742 - acc: 0.7022 - val_loss: 0.9206 - val_acc: 0.6480
Epoch 13/50
25/25 [=====] - 20s 794ms/step - loss:
0.7367 - acc: 0.7154 - val_loss: 0.7945 - val_acc: 0.6929
Epoch 14/50
25/25 [=====] - 19s 778ms/step - loss:
0.7020 - acc: 0.7354 - val_loss: 0.7705 - val_acc: 0.6948
Epoch 15/50
25/25 [=====] - 20s 807ms/step - loss:
0.6742 - acc: 0.7325 - val_loss: 0.7577 - val_acc: 0.7140
Epoch 16/50
```

```
25/25 [=====] - 20s 781ms/step - loss:
0.6758 - acc: 0.7418 - val_loss: 0.7766 - val_acc: 0.7131
Epoch 17/50
25/25 [=====] - 20s 810ms/step - loss:
0.6605 - acc: 0.7466 - val_loss: 0.7200 - val_acc: 0.7269
Epoch 18/50
25/25 [=====] - 20s 793ms/step - loss:
0.6453 - acc: 0.7500 - val_loss: 0.7373 - val_acc: 0.7186
Epoch 19/50
25/25 [=====] - 20s 808ms/step - loss:
0.6381 - acc: 0.7585 - val_loss: 0.7292 - val_acc: 0.7269
Epoch 20/50
25/25 [=====] - 20s 795ms/step - loss:
0.6110 - acc: 0.7673 - val_loss: 0.7906 - val_acc: 0.7131
Epoch 21/50
25/25 [=====] - 20s 795ms/step - loss:
0.6187 - acc: 0.7629 - val_loss: 0.7009 - val_acc: 0.7296
Epoch 22/50
25/25 [=====] - 20s 797ms/step - loss:
0.5574 - acc: 0.7865 - val_loss: 0.6847 - val_acc: 0.7599
Epoch 23/50
25/25 [=====] - 20s 799ms/step - loss:
0.5884 - acc: 0.7788 - val_loss: 0.6892 - val_acc: 0.7415
Epoch 24/50
25/25 [=====] - 20s 796ms/step - loss:
0.5553 - acc: 0.7869 - val_loss: 0.7059 - val_acc: 0.7360
Epoch 25/50
25/25 [=====] - 20s 797ms/step - loss:
0.5731 - acc: 0.7867 - val_loss: 0.6628 - val_acc: 0.7479
Epoch 26/50
25/25 [=====] - 20s 799ms/step - loss:
0.5141 - acc: 0.7989 - val_loss: 0.6582 - val_acc: 0.7681
Epoch 27/50
25/25 [=====] - 20s 800ms/step - loss:
0.4982 - acc: 0.8122 - val_loss: 0.8243 - val_acc: 0.7232
Epoch 28/50
25/25 [=====] - 20s 801ms/step - loss:
0.5484 - acc: 0.7898 - val_loss: 0.6767 - val_acc: 0.7562
Epoch 29/50
25/25 [=====] - 20s 800ms/step - loss:
0.4928 - acc: 0.8129 - val_loss: 0.7491 - val_acc: 0.7498
Epoch 30/50
25/25 [=====] - 20s 799ms/step - loss:
0.4768 - acc: 0.8238 - val_loss: 0.6758 - val_acc: 0.7544
Epoch 31/50
25/25 [=====] - 20s 795ms/step - loss:
0.4678 - acc: 0.8219 - val_loss: 0.7470 - val_acc: 0.7415
Epoch 32/50
```

```
25/25 [=====] - 20s 798ms/step - loss:
0.4604 - acc: 0.8260 - val_loss: 0.6842 - val_acc: 0.7489
Epoch 33/50
25/25 [=====] - 20s 796ms/step - loss:
0.4755 - acc: 0.8263 - val_loss: 0.7710 - val_acc: 0.7589
Epoch 34/50
25/25 [=====] - 20s 800ms/step - loss:
0.4409 - acc: 0.8345 - val_loss: 0.6872 - val_acc: 0.7782
Epoch 35/50
25/25 [=====] - 20s 807ms/step - loss:
0.4372 - acc: 0.8378 - val_loss: 0.6577 - val_acc: 0.7626
Epoch 36/50
25/25 [=====] - 20s 810ms/step - loss:
0.3909 - acc: 0.8534 - val_loss: 0.6526 - val_acc: 0.7910
Epoch 37/50
25/25 [=====] - 20s 784ms/step - loss:
0.3928 - acc: 0.8558 - val_loss: 0.7110 - val_acc: 0.7663
Epoch 38/50
25/25 [=====] - 20s 793ms/step - loss:
0.3955 - acc: 0.8530 - val_loss: 0.7136 - val_acc: 0.7773
Epoch 39/50
25/25 [=====] - 20s 808ms/step - loss:
0.3658 - acc: 0.8659 - val_loss: 0.6593 - val_acc: 0.7773
Epoch 40/50
25/25 [=====] - 20s 798ms/step - loss:
0.3916 - acc: 0.8588 - val_loss: 0.6378 - val_acc: 0.7782
Epoch 41/50
25/25 [=====] - 20s 797ms/step - loss:
0.3722 - acc: 0.8565 - val_loss: 0.7199 - val_acc: 0.7736
Epoch 42/50
25/25 [=====] - 20s 782ms/step - loss:
0.3746 - acc: 0.8533 - val_loss: 0.6931 - val_acc: 0.7791
Epoch 43/50
25/25 [=====] - 20s 795ms/step - loss:
0.3255 - acc: 0.8787 - val_loss: 0.6562 - val_acc: 0.8029
Epoch 44/50
25/25 [=====] - 20s 809ms/step - loss:
0.3306 - acc: 0.8738 - val_loss: 0.7102 - val_acc: 0.7635
Epoch 45/50
25/25 [=====] - 20s 780ms/step - loss:
0.2981 - acc: 0.8809 - val_loss: 0.7033 - val_acc: 0.7929
Epoch 46/50
25/25 [=====] - 20s 809ms/step - loss:
0.3009 - acc: 0.8903 - val_loss: 0.6488 - val_acc: 0.8066
Epoch 47/50
25/25 [=====] - 20s 783ms/step - loss:
0.3078 - acc: 0.8829 - val_loss: 0.6670 - val_acc: 0.7965
Epoch 48/50
```

```

25/25 [=====] - 20s 809ms/step - loss:
0.2886 - acc: 0.8906 - val_loss: 0.7584 - val_acc: 0.7828
Epoch 49/50
25/25 [=====] - 20s 797ms/step - loss:
0.3103 - acc: 0.8860 - val_loss: 0.6943 - val_acc: 0.7809
Epoch 50/50
25/25 [=====] - 20s 808ms/step - loss:
0.3007 - acc: 0.8898 - val_loss: 0.7433 - val_acc: 0.7754

```

After the compilation we can see the result of our model which is accuracy is 88.98% and Val. Acc is 77.54% which can be increased if provide more data for training or do some changes with the model by changing its optimizers or activation function or learning rate.

Step 8: Viewing the model performance

Now we will plot our trained model results in a graph with the help of matplotlib.

```

plt.plot (History.history['loss'])
plt.plot (History.history['val_loss'])
plt.title ('Model Loss')
plt.ylabel ('Loss')
plt.xlabel ('Epochs')
plt.legend (['train', 'test'])
plt.show ()

```

Output: as in Fig. 11.2

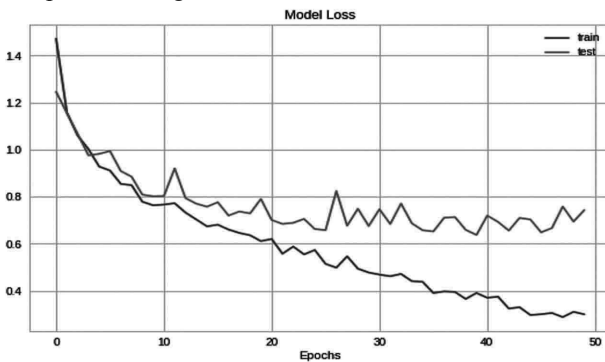


Fig. 11.2 Loss Representation of Model

```

plt.plot (History.history['acc'])
plt.plot (History.history['val_acc'])
plt.title ('Model Accuracy')
plt.ylabel ('Accuracy')
plt.xlabel ('Epochs')
plt.legend (['train', 'test'])
plt.show ()

```

Output: as in Fig. 11.3

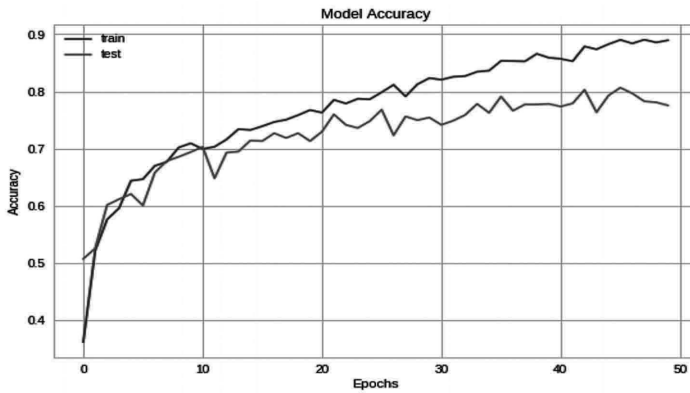


Fig. 11.3 Accuracy Representation of Model

Step 9: Prediction on validation data

Here we will get our trained model to predict the images with the validation data which we have separated while labelling and splitting our data.

```
# getting predictions on val set.
pred=model.predict (x_test)
pred_digits=np.argmax (pred,axis=1)

# now storing some properly as well as misclassified indexes.
i=0
prop_class=[]
mis_class=[]
```

Here we are importing images from the validation data to test the model for its prediction.

```
for i in range (len (y_test)):
    if (np.argmax (y_test[i])==pred_digits[i]):
        prop_class.append (i)
    if (len (prop_class)==8):
        break

i=0
for i in range (len (y_test)):
    if (not np.argmax (y_test[i])==pred_digits[i]):
        mis_class.append (i)
    if (len (mis_class)==8):
        break

warnings.filterwarnings ('always')
warnings.filterwarnings ('ignore')
count=0
fig,ax=plt.subplots (4,2)
fig.set_size_inches (15,15)
for i in range (3):
    for j in range (2):
```

```

ax[i,j].imshow (x_test[prop_class[count]])
ax[i,j].set_title ("Predicted Flower : "+str (le.inverse_
transform ([pred_digits[prop_class[count]]]))+"\n"+"Actual
Flower : "+str (le.inverse_transform (np.argmax ([y_
test[prop_class[count]]])))
plt.tight_layout ()
count+=1

```

Output: as in Fig. 11.4

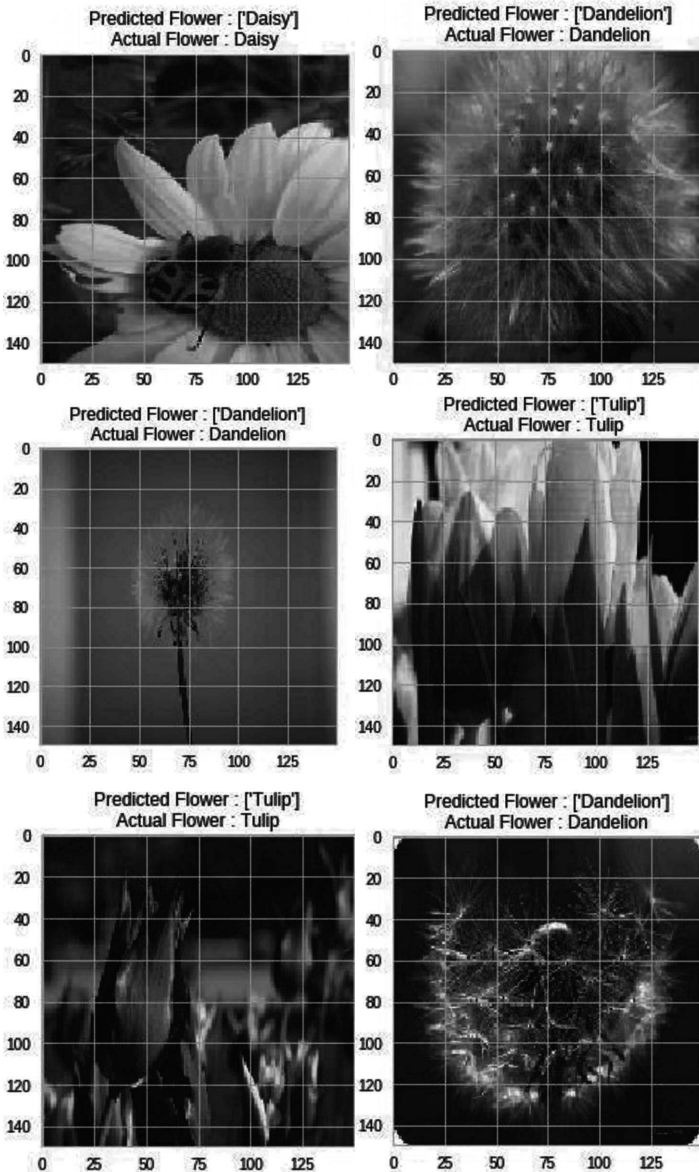


Fig. 11.4 Prediction of flowers**Step 10: Saving the model**

This is the way to save a model which we have trained

```
model_json = model.to_json ()
with open ("model.json", "w") as json_file:
    json_file.write (model_json)
# serialize weights to HDF5
model.save_weights ("model.h5")
print ("Model Saved")
```

After saving this model we can predict the values under these categories just by using the images of those species.

Summary

Species Recognition also plays an important role in the field of agriculture which can help a lot of people to identify the plant species digitally and it will last long with a guarantee that our system is correct to identify a species of plant. In this chapter we have worked on one more project which is related to species recognition and we can totally understand how to use the dataset with machine learning, deep learning and computer vision for better understand and real time use of artificial Intelligence. In the Species recognition we have used flower dataset to train our model for classification and identification of flowers in Realtime. This project can be used in Realtime to identify the species of flowers digitally by just from images of flowers

Bibliography

- Rajesh Singh, Anita Gehlot, Sushabhan Choudhury, Bhupendra Singh, “Embedded System based on ATmega Microcontroller-Simulation, Interfacing and Projects”, Narosa Publishing House, 2017, ISBN: 978-81-8487-5720.
- Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Arduino-Based Embedded Systems: Interfacing, Simulation, and LabVIEW GUI”- CRC Press (Taylor & Francis), 2017, ISBN 9781138060784.
- Anita Gehlot, Rajesh Singh, Devender Kumar Saini, Monika Yadav, Bhupendra Singh, “IoT Enabled Smart Microgrid.....Rapid Prototyping”, GBS Publication, 2018, ISBN: 978-93-87374-41-6.
- Anita Gehlot, Rajesh Singh, Mamta Mittal, Bhupendra Singh, Ravindra Sharma, Vikas Garg, “Hands on approach on Applications of Internet of Things in various Domain of Engineering”, International Research Publication House, 2018, ISBN: 978-93-87385-25-3.
- Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Internet of Things Enabled Automation in Agriculture”, New India Publishing Agency, 2018, ISBN: 9789387973053.
- Rajesh Singh, Anita Gehlot, Bhupendra Singh, Bikarama Prasad Yadav, “IoT Enabled Fire Safety and Security Devices for Building”, Pen2Print, EduPedia Publications, 2018, ISBN:9789386647979.

- Rajesh Singh, Anita Gehlot, Bhupendra Singh, Piyush Kuchhal, "Wireless Methods and Devices for Home Automation" International Research Publication House, 2018, ISBN: 978-93-87388-27-7.
- Rajesh Singh, Anita Gehlot, Raghuveer Chimata, Bhupendra Singh, P.S.Ranjith, "Internet of Things in Automotive Industries and Road Safety" River Publishers, 2018, ISBN:9788770220101&e-ISBN:9788770220095
- Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, "Arduino meets MATLAB.... Interfacing, Programs and Simulink", Bentham Science, 2018, ISBN: 978-1-68108-728-3 & e-ISBN: 978-1-68108-727-6
- Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Bhupendra Singh, and Priyanka Tyagi, "Getting Started for Internet of Things with Launch Pad and ESP8266", River Publishers, 2019, ISBN: 9788770220682, e-ISBN: 9788770220675
- Rajesh Singh, Anita Gehlot, Bhupendra Singh, "Arduino and Scilab based Projects", Bentham Science, 2019, ISBN: 9789811410918 & e-ISBN: 9789811410925
- Anita Gehlot, Rajesh Singh, Lovi Raj Gupta, Bhupendra Singh, "Cook Book for Mobile Robotic Platform Control with Internet of Things and Ti Launch Pad", BPB Publisher, 2019, ISBN: 978-93-8851-167-4
- Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Bhupendra Singh, Mahindra Swain, "Internet of Things with Raspberry Pi and Arduino", CRC press/Taylor & Francis, 2019, ISBN: 9780367248215
- Anita Gehlot, Rajesh Singh, Gaurav Sethi, Bhupendra Singh, "The Robotic Platform Control with Atmega328 and NuttyFi Based on the Internet of Things", Nova Science Publisher, 2020, ISBN:9781536174724
- Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Navjot Rathour, Mahendra Swain, Bhupendra Singh, "IoT based projects", BPB publication, 2020, ISBN: 9789389328523